

- Smooth experience while watching 120fps visuals as they might well operate under 8ms.

Probable Hazards

Unexpected pitfalls might appear in the form of not-so-smooth UI or while tuning the performance of the application. In these cases, the Flutter Plugin available on the IDE will come to the rescue by enabling the Show Widget Rebuild Information checkbox which helps in tracking down frames that are being displayed and rendered in more than 16ms. According to the capacity, the plugin also provides a relevant link.

The following actions might help in negating the possible pitfalls:

- Avoid using the Opacity widget, and particularly avoid it in an animation. AnimatedOpacity or FadeInImage should be put to use instead of the Opacity widget.
- When using an AnimatedBuilder, a subtree in the builder function that builds widgets independent on the animation should not be installed. This sub-tree is rebuilt for every bit of the concerned animation. Instead, that part of the sub-tree should be constructed once and passed on as a descendant to the AnimatedBuilder.
- Pre-clipping an image before animating is advisable and clipping should be strictly avoided in an animation.
- Constructors with a complete view of the descendants should be avoided in usage because most of the children are not present on-screen to optimise the generation cost. 